

Manhattan Galleries — Project State

Snapshot from May 15, 2026 session · NYC-Gallery-App on GitHub

The project in one paragraph

A static map of Manhattan with one dot per art gallery. Dots are colored by whether the gallery's website has changed since the last scrape (orange = updated, gray = no change). Hosted entirely on GitHub: GitHub Pages serves the static *index.html*; a GitHub Actions cron job runs *scraper.py* daily, writes results to *data/galleries.json*, and GalleryBot commits the update.

Repo: github.com/JF11579/NYC-Gallery-App (public)

Where we stopped — immediate next step

Claude wrote a one-time script called **build_gallery_list.py** that fetches three gallery directories (downtowngallerymap.com LES + Tribeca/SoHo, and agora-gallery.com Upper East Side), parses out gallery names, addresses, and website URLs, dedupes them, geocodes via Mapbox, and writes a populated *data/galleries.json* in GeoJSON format.

Joe needs to run this script **once, locally, on the Sea Cliff Linux Mint desktop**. The file has been downloaded but its location on the machine is unclear — last terminal output showed it wasn't at *~/Desktop/gallery_build/* or *~/build_gallery_list.py*.

Step 1: find the file

Open a terminal and run:

```
ls ~/Downloads/build_gallery_list*
```

If it's there (most likely place if downloaded via Firefox), continue to Step 2. If not, search wider:

```
find ~ -name 'build_gallery_list*' 2>/dev/null
```

Step 2: move it into a working folder and dry-run

Once located, set up the working folder and execute a dry run (no Mapbox calls):

```
mkdir -p ~/Desktop/gallery_build
mv ~/Downloads/build_gallery_list.py ~/Desktop/gallery_build/
cd ~/Desktop/gallery_build
python3 build_gallery_list.py --dry-run
```

Expected output: three pages fetched, ~150–400 galleries parsed across LES, SoHo/Tribeca, and UES, then a list printed. Send this output to Claude before running for real.

(*requests* and *beautifulsoup4* are already installed on this machine — the earlier *pip install* confirmed both were present.)

Step 3: real run with Mapbox geocoding

Only after the dry run looks reasonable:

```
export MAPBOX_TOKEN='pk.eyJ1IjoiamYxMTU3OS...' # token from index.html
python3 build_gallery_list.py
```

This will take 3–5 minutes — one geocoding call per gallery, polite pacing. Output appears at *~/Desktop/gallery_build/data/galleries.json*.

Important: Do not trigger the daily GitHub Actions workflow after pushing the new *galleries.json* until Claude has provided the Stage 2 scraper. The current *scraper.py* in the repo writes one hardcoded gallery and would overwrite the full list. Order: (1) push new *galleries.json*, (2) get Stage 2 scraper from Claude, (3) push that, (4) only then trigger workflow.

Current state of components

Working

- GitHub Actions cron schedule (daily 12:00 UTC) is wired and committing as GalleryBot.
- Mapbox token in *index.html* works; map renders the light-v11 base style centered on Chelsea.
- Stage 1 code is written but **not yet deployed** — see below.
- *build_gallery_list.py* is written and tested against synthetic fixtures; parsers correctly handle Broadway/Bowery addresses, ordinal addresses like '45 East 78th Street', and rejects false positives like 'Salon 94'.

Not working / not yet done

- **Stage 1 not deployed.** Claude wrote updated *scraper.py* (GeoJSON format) and *index.html* (orange/gray conditional coloring + legend + popup) but Joe has not pushed them to the repo yet. As a result, the map currently shows nothing — the existing *scraper.py* writes plain JSON instead of GeoJSON, which Mapbox can't render.
- **Gallery list not built.** Pending the *build_gallery_list.py* run described above.
- **Stage 2 scraper not written.** This is the real change-detection scraper — fetches each gallery URL, hashes the page, compares to a stored hash, flips *updated: true/false*. Will be written after the gallery list is in place.

Decisions locked in

- **Colors:** orange (#f57c00) for updated, gray (#9e9e9e) for no change. Colorblind-friendly.
- **"Updated" definition:** any change since the last (24-hour) scrape.
- **Hash storage:** separate *data/hashes.json* file in the repo (decided but not yet implemented).
- **Sources for the gallery list:** downtowngallerymap.com (LES + Tribeca/SoHo) and agora-gallery.com (UES). Skips Chelsea for v1 — Joe's call, on the basis that downtown + UES covers the bulk of meaningful gallery activity.
- **Geocoding:** Mapbox forward geocoding API, biased toward Manhattan, free tier (100k requests/month).
- **Coordinate format:** [lon, lat] (Mapbox convention).
- **Run location:** Sea Cliff Linux Mint desktop, terminal (not Jupyter, not Colab).

What comes after (preview)

Once *galleries.json* is populated and pushed, Claude will write the Stage 2 scraper, which will:

- Read the existing *galleries.json* (preserving the gallery list)
- For each gallery URL, fetch the page and compute a content hash
- Compare against the previous hash stored in *data/hashes.json*
- Flip *properties.updated* to true or false based on whether the hash changed
- Write back both files; GalleryBot commits as usual

Stage 3 (later): polish, error handling for galleries whose sites are offline, maybe a small "last updated X days ago" indicator in popups.

Files Claude generated in this session

- **scraper.py** (Stage 1) — placeholder that writes one gallery (303 Gallery) in correct GeoJSON format with *updated: true* hardcoded. For testing the render pipeline only.
- **index.html** — updated map with orange/gray color logic, legend, hover cursor, richer popup, and a cache-busted data URL.
- **build_gallery_list.py** — the one-time gallery list builder. This is the file we need to run next.

All three are available in the May 15 Claude chat. The *build_gallery_list.py* file is the immediate priority.

Parking lot — not urgent

- Add Chelsea coverage once v1 is shipped (~200 more galleries, would push total toward 500).
- Tighten 56 Henry's address — parser currently picks '105 Henry Street' from the listing '56 / 105 Henry Street'. Geocodes to roughly the right spot anyway.
- Mapbox token in *index.html* is a public token (intentionally exposed in client code), but if rotation is desired it can be regenerated in Mapbox dashboard.